# A Dataset Details

We evaluate OLMoE on four domains used during pretraining: **math**, **code**, **commonsense**, and **domain-specific** (scientific text). For each domain, we construct three sets:

- **Pretraining sample set:** 10,000 examples drawn from OLMoE's pretraining data.
- **Test sample set:** 10,000 examples from a benchmark dataset in the same domain, following OLMoE's categorization where available.
- **Validation set:** a disjoint dataset used solely to compute validation loss, enabling correlation analysis between in-domain learning and test generalization.

For the domain-specific case, we filter the scientific pretraining data using the keyword *"Organic Chemistry"*, aligning the training distribution with the "college_chemistry" benchmark in MMLU.

Table 2 summarizes all dataset sources. All pretraining sets are directly drawn from OLMoE's pretraining corpora; test benchmarks are standard evaluation datasets; validation sets are only used for correlation analysis.

Table 2: Datasets used in our domain-level evaluation. Pretraining samples are from OLMoE corpora, test benchmarks are used for evaluation, and validation sets are disjoint corpora used only for loss–accuracy correlation. All datasets accessed via HuggingFace.

| Domain | Pretraining Source | Test Dataset | Validation Dataset |
|---|---|---|---|
| Math | `open-web-math/open-web-math` | `hkust-nlp/dart-math-pool-math` | `qwedsacf/competition_math` |
| Code | `bigcode/starcoderdata` | `evalplus/humanevalplus` | `newfacade/LeetCodeDataset` |
| Commonsense | `emozilla/dolma-v1_7-books` | `tau/commonsense_qa` | `kowndinya23/flan2021-commonsense` |
| Domain-Specific | `blester125/mediawiki-dolma`(filtered "Organic Chemistry") | `cais/mmlu` ("college_chemistry" subset) | `chemNLP/chemistry-bookshelves-merged` |

# B Experiment Details

**Compute Resources.** All experiments are conducted on NVIDIA A100 GPUs with 80GB memory, using public OLMoE checkpoints.

**Instruction Tuning.** The raw OLMoE checkpoints exhibit weak instruction-following ability, making direct evaluation of generalization behavior unreliable—particularly in text-based domains such as commonsense reasoning or code completion. To address this, we apply light instruction tuning using LoRA [9] to make the pretrained model *follow prompts in a human-readable way*. This ensures we can evaluate generalization while preserving the original pretraining behavior.

To avoid overriding pretraining representations, we adopt minimal finetuning using a small number of epochs and low-rank parameter updates. Specifically:

- **Math:** Finetune with LoRA for 3 epochs on the same math SFT dataset `meta-math/MetaMathQA` used by OLMoE.
- **Code:** Finetune for 3 epochs on `HuggingFaceH4/CodeAlpaca_20K`.
- **Commonsense & Domain-Specific:** Finetune for 3 epochs on `yahma/alpaca-cleaned`.

Key configurations of LoRA include:

- **Target modules:** `q_proj`, `k_proj`, `v_proj`
- **LoRA rank:** 32
- **LoRA dropout:** 0.1
- **Learning rate:** 5e-5

- **Batch size:** 4
- **Max sequence length:** 2048

**Applying Min-K%++ for Training-Test Separation.** To maintain a rigorous separation between the pretraining and test datasets, we employ the Min-K%++ membership inference attack [35], a state-of-the-art, reference-free method designed to detect whether specific data samples are part of a model's training corpus.

Specifically, we apply the Min-K%++ method to the test dataset to identify samples that the model is most likely to have encountered during training. Determining an optimal threshold for classification is challenging due to variability in model behaviors and data distributions. To address this, we adopt a conservative approach by removing the top 10% of test samples with the highest Min-K%++ scores, thereby minimizing the risk of training data contamination in the test set.

This strategy ensures that our evaluation metrics accurately reflect the model's generalization capabilities on truly unseen data, enhancing the validity of our experimental results.

### B.1 Construction of Training–Test Paired Groups.

Here we provide additional details on how we construct the training–test paired groups used in Figure 2 to analyze the temporal relation between memorization and generalization. The overall procedure is introduced in Section 3.2; here, we summarize the specific grouping and matching setup for clarity and reproducibility.

**Training groups.** For each pretraining sample, we track its token-level loss across checkpoints and mark the earliest step $t_i^*$ when the loss remains below a threshold $\varepsilon$ and deviates by less than $\delta$ from its final value for all later checkpoints. Samples sharing the same $t_i^*$ form a *training group* denoted as *Pretrain*@$t_i^*$. For $(\varepsilon, \delta)$, $\delta$ is fixed at 0.05 across domains, while $\varepsilon$ is domain-specific and determined based on the average loss scale of each domain. We test $\varepsilon$ in increments of 0.1 to ensure that the cumulative proportion of memorized samples before the final checkpoint remains between 20–25% across domains, balancing sufficient data coverage with stable per-domain scaling. Ablation studies on the robustness of $(\varepsilon, \delta)$ is provided in Appendix E.3.

**Test groups.** Each test sample's accuracy trajectory is recorded across checkpoints. A test sample is considered to have generalized at the earliest step $t_j^\#$ where its accuracy remains consistently correct thereafter, with mean accuracy above 0.8 and at most one error in the following checkpoints. Samples sharing the same $t_j^\#$ form a *test group* denoted as *Test*@$t_j^\#$.

**Embedding-based pairing.** To align semantically related training and test groups, we compute sentence embeddings for all samples using the SentenceTransformer model "all-MiniLM-L6-v2" and measure pairwise cosine similarity between groups within each domain. The average similarity between every pair of groups forms a similarity matrix, and a one-to-one mapping between training and test groups is obtained using the Hungarian algorithm implemented in SciPy, which maximizes overall semantic similarity. The notation *Pretrain*@$p \rightarrow$ *Test*@$q$ thus indicates that the training group memorized at step $p$ is semantically aligned with a test group whose performance stabilizes at step $q$.

## C Generalization Bound for MoE with Routing Kernel

### C.1 Model Setup and Assumptions

Consider a MoE model where the router is fixed after pretraining, and the expert networks are trainable. Let $K$ be the total number of experts in the model.

- $f_k(\phi_k, x) \in \mathbb{R}$ be the $k$-th expert's output, with trainable parameters $\phi_k$.
- $g_k(x) \in [0, 1]$ be the $k$-th routing weight, which is fixed and pre-determined for any input $x$. We assume $\sum_{k=1}^K g_k(x) = 1$ (e.g., from a fixed softmax layer or pre-computed weights).
- Full model: $F(\theta, x) = \sum_{k=1}^K g_k(x) f_k(\phi_k, x)$, where $\theta = (\phi_1, \ldots, \phi_K)$ is the collection of all trainable expert parameters.

**Key Assumptions:**

1. **NTK Regime for Experts:** Expert parameters $\theta = (\phi_1, \ldots, \phi_K)$ are initialized as $\phi_k(0) \sim \mathcal{N}(0, I)$ for each $k$, and remain $\epsilon$-close to initialization ($\|\theta(t) - \theta(0)\| = O(1/\sqrt{\text{width}})$), enabling linearization via Neural Tangent Kernel (NTK). We assume $\phi_k(0)$ are independent across different experts $k$.
2. **Fixed Routing:** The routing weights $g_k(x)$ are fixed for all inputs $x$ and do not contain any trainable parameters. They satisfy $\max_k \|g_k\|_\infty \leq 1$.
3. **Label Noise:** Training labels $y_i = f^*(x_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, independent of $x_i$.

### C.2 Routing Kernel Definition

Define the NTK for this MoE configuration. Consistent with the general routing kernel definition in the main text, $\Theta_{\text{route}}(x, x')$ here is specifically instantiated as:

$$\begin{aligned}\Theta_{\text{route}}(x, x') &= \mathbb{E}_{\theta(0)}\left[\left\langle \frac{\partial F(\theta(0), x)}{\partial \theta}, \frac{\partial F(\theta(0), x')}{\partial \theta} \right\rangle\right] \\ &= \sum_{j=1}^{K} g_j(x) g_j(x') \mathbb{E}_{\phi_j(0)}\left[\left\langle \frac{\partial f_j(\phi_j(0), x)}{\partial \phi_j}, \frac{\partial f_j(\phi_j(0), x')}{\partial \phi_j} \right\rangle\right]\end{aligned}$$

Let $K_{f_j}(x, x') = \mathbb{E}_{\phi_j(0)}\left[\left\langle \frac{\partial f_j(\phi_j(0), x)}{\partial \phi_j}, \frac{\partial f_j(\phi_j(0), x')}{\partial \phi_j} \right\rangle\right]$ be the NTK for the $j$-th expert network. Then, the Routing Kernel in this fixed-routing scenario is:

$$\Theta_{\text{route}}(x, x') = \sum_{j=1}^{K} g_j(x) g_j(x') K_{f_j}(x, x')$$

The structure of $\Theta_{\text{route}}(x, x')$ in this fixed-routing setup provides crucial insights into how routing influences generalization. It is composed of two primary factors: the gating weight product $g_j(x)g_j(x')$, and the individual expert NTKs $K_{f_j}(x, x')$. The term $g_j(x)g_j(x')$ signifies the importance of the $j$-th expert for both inputs $x$ and $x'$, highlighting that two inputs are considered "similar" by the kernel if they are significantly routed to the *same* experts. If inputs $x$ and $x'$ are routed to distinct sets of experts, their contribution to the sum for a given $j$ will be diminished. Simultaneously, $K_{f_j}(x, x')$ captures the functional similarity learned by the $j$-th expert for inputs $x$ and $x'$. Thus, the overall routing kernel measures a compounded similarity: inputs are similar not only if they are guided through similar pathways (weighted by $g_j(x)g_j(x')$), but also if the specific experts they pass through exhibit similar functional behavior for those inputs ($K_{f_j}(x, x')$). This decomposition underscores how fixed routing patterns (reflected in $g_j(x)$) set the stage for expert specialization and ultimately shape the generalization properties of the MoE model.

### C.3 Main Theorem

**Theorem C.1** (Generalization Bound for MoE Experts)**.** *Under Assumptions 1-3, with probability $1 - \delta$ over training samples $(x_i, y_i)_{i=1}^n$, the excess risk satisfies:*

$$\begin{aligned}\mathbb{E}_{x,y}\left[(F(\theta^*, x) - f^*(x))^2\right] \leq & \underbrace{\frac{C_1 \lambda^2}{n} \boldsymbol{y}^\top (\boldsymbol{H}^* + \lambda I)^{-2} \boldsymbol{y}}_{\textit{Bias}} \\ & + \underbrace{C_2 \left( \frac{\sigma^2 Tr(\boldsymbol{H}^* (\boldsymbol{H}^* + \lambda I)^{-1})}{n} + \frac{\sigma^2 \log(1/\delta)}{n} \right)}_{\textit{Variance + Noise}}\end{aligned}$$

*where $\boldsymbol{H}^*_{i,j} = \Theta_{\text{route}}(x_i, x_j)$, $\lambda > 0$ is regularization, and $C_1, C_2$ are constants depending on the properties of $g_k(x)$ and $f_k(x)$. The bound decays as $O\left(\frac{1}{n}\right)$ when $\lambda = \Theta(1)$.*